

Dados inteiros

Tipo de dado	Intervalo	Tamanho
int	-32.768 até 32.767	16 bits
int	-2.147.483.648 até 2.147.483.647	32 bits
short	-32.768 até 32.767	16 bits
short int	-32.768 até 32.767	16 bits
signed int	-32.768 até 32.767	16 bits
unsigned int	0 até 65.535	16 bits
signed short int	-32.768 até 32.767	16 bits
unsigned short int	0 até 65.535	16 bits
long	-2.147.483.648 até 2.147.483.647	32 bits
long int	-2.147.483.648 até 2.147.483.647	32 bits
signed long int	-2.147.483.648 até 2.147.483.647	32 bits
unsigned long int	0 até 4.292.967.295	32 bits

15 de 64

Dados reais

Tipo de dado	Intervalo	Tamanho
float	3.4e-38 até 3.4e+38	32 bits
double	-1.7e-308 até 1.7e+308	64 bits
long double	3.4e-4932 até 1.1e+4932	80 bits

Dados caracteres

Tipo de dado	Intervalo	Tamanho
char / signed char	-127 até 127	8 bits
unsigned char	0 até 255	8 bits

20 de 64

Dados lógicos

- Os dados lógicos, ou booleanos, ocupam apenas 1 byte de memória.
- Eles são referenciados pela palavra-chave `bool` e podem possuir apenas dois tipos de valores: `true` ou `false`.
- Os dados lógicos são utilizados para verificar o resultado de uma condição ou expressão lógica.

21 de 64

Operadores aritméticos

Operador	Operação	Tipo	Resultado
+	Manutenção de sinal	Unário	...
-	Inversão de sinal	Unário	...
%	Resto de divisão (módulo)	Binário	Inteiro
/	Divisão	Binário	Inteiro ou Real
*	Multiplicação	Binário	Inteiro ou Real
+	Adição	Binário	Inteiro ou Real
-	Subtração	Binário	Inteiro ou Real
++	Incremento	Unário	Inteiro ou Real
--	Decremento	Unário	Inteiro ou Real

22 de 64

Instruções da linguagem C++

- As instruções são representadas pelo conjunto de palavras-chave de uma linguagem de programação.
- A linguagem C++ diferencia caracteres maiúsculos e minúsculos.
- Nenhum comando da linguagem pode ser escrito com caracteres maiúsculos.
- Nenhuma variável, constante ou função pode ser definida utilizando um dos nomes reservados para os comandos da linguagem de programação.

23 de 64

Estrutura de um programa em C++

```
/* Programa 01 */  
// programa01.cpp  
  
#include <iostream>  
using namespace std;  
  
// Função principal  
int main() {  
    cout << "Hello World!\n";  
    return 0;  
}  
  
// Corpo das funções
```

Comentários

Diretivas de pré-processamento, cabeçalhos, variáveis globais, protótipos de funções, classes, etc.

Corpo da função principal

Corpo das funções

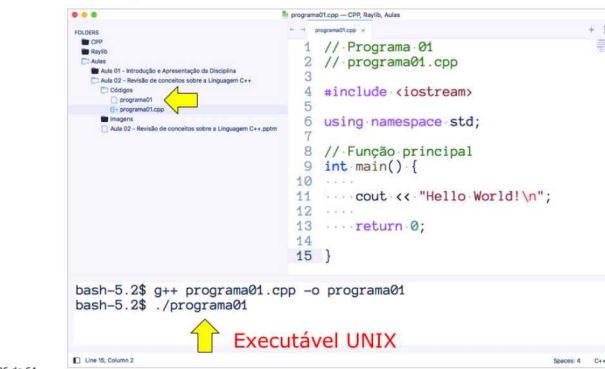
24 de 64

Exemplo editado no Sublime Text 4



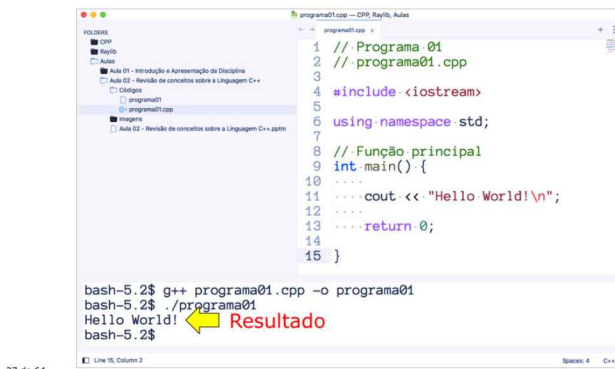
25 de 64

Exemplo editado no Sublime Text 4



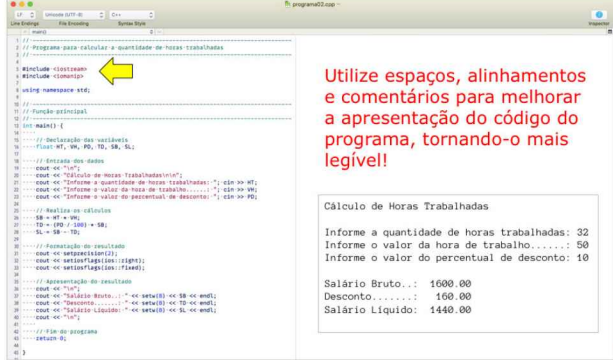
26 de 64

Exemplo editado no Sublime Text 4



27 de 64

Exemplo de formatação do código



Utilize espaços, alinhamentos e comentários para melhorar a apresentação do código do programa, tornando-o mais legível!

Cálculo de Horas Trabalhadas

Informe a quantidade de horas trabalhadas: 32
Informe o valor da hora de trabalho.....: 50
Informe o valor do percentual de desconto: 10

Salário Bruto.: 1600.00
Desconto.....: 160.00
Salário Líquido: 1440.00

28 de 64

Formatação de valores numéricos: **ios::**

Sinalizador	Finalidade
<code>ios::dec</code>	Formata a saída, utilizando o formato decimal.
<code>ios::fixed</code>	Formata a saída, utilizando o formato decimal.
<code>ios::hex</code>	Formata a saída, utilizando o formato hexadecimal.
<code>ios::oct</code>	Formata a saída, utilizando o formato octal.
<code>ios::left</code>	Formata a saída, justificando os caracteres à esquerda.
<code>ios::right</code>	Formata a saída, justificando os caracteres à direita.
<code>ios::scientific</code>	Formata a saída, utilizando a notação científica.
<code>ios::showbase</code>	Formata a apresentação de valores octais e hexadecimais.
<code>ios::showpoint</code>	Força a apresentação de um ponto decimal em um valor real.
<code>ios::showpos</code>	Exibe o sinal unário de um valor numérico (positivo/negativo).
<code>ios::uppercase</code>	Exibe os caracteres, utilizando letras maiúsculas.

29 de 64

Operadores relacionais

Símbolo	Significado
<code>==</code>	Igual a
<code>!=</code>	Diferente de
<code>></code>	Maior que
<code><</code>	Menor que
<code>>=</code>	Maior ou igual que
<code><=</code>	Menor ou igual que

30 de 64

Desvio condicional simples: **if**

```
if (condição) {  
    // Instruções executadas  
    // quando a condição  
    // for verdadeira  
}  
  
// Instruções executadas após a  
// condição ser verdadeira ou falsa
```

31 de 64

Desvio condicional composto: **if-else**

```
if (condição) {  
    // Instruções executadas quando  
    // a condição for verdadeira  
}  
else {  
    // Instruções executadas quando  
    // a condição for falsa  
}  
  
// Instruções executadas após a  
// condição ser verdadeira ou falsa
```

32 de 64

Desvio condicional aninhado

```
if (condição externa) {  
    // Instruções executadas quando a condição for verdadeira  
    if (condição interna) {  
        // Instruções executadas quando a condição interna for verdadeira  
    }  
    else {  
        // Instruções executadas quando a condição interna for falsa  
    }  
}  
else {  
    // Instruções executadas quando a condição externa for falsa  
}  
  
// Instruções executadas após a condição externa ser verdadeira ou falsa
```

33 de 64

Operador ternário

- O operador ternário representa uma maneira mais simplificada de escrever uma expressão condicional que seleciona um dos dois valores com base em uma condição booleana. Ele é útil para simplificar o código de uma instrução *if-else*.
- O operador ternário possui a seguinte sintaxe:

condição ? expressão1 : expressão2;

↑ Valor para a condição verdadeira ↑ Valor para a condição falsa

Operador lógico de conjunção: **and** ou **&&**

Condição 1	Condição 2	Resultado
Falsa	Falsa	Falso
Verdadeira	Falsa	Falso
Falsa	Verdadeira	Falso
Verdadeira	Verdadeira	Verdadeiro

35 de 64

```
// Verifica se o jogador atingiu uma região  
if (pos.x >= 300 && pos.y == 5) {  
    // Instruções...  
}
```

Operador lógico de disjunção inclusiva: **or** ou **||**

Condição 1	Condição 2	Resultado
Falsa	Falsa	Falso
Verdadeira	Falsa	Verdadeiro
Falsa	Verdadeira	Verdadeiro
Verdadeira	Verdadeira	Verdadeiro

36 de 64

```
// Verifica a colisão com as bordas laterais  
if (ball.x <= 0 || ball.x >= GetScreenWidth()) {  
    // Instruções...  
}
```

▪ Operador lógico de negação: **not** ou **!**

Condição	Resultado
Verdadeira	Falso
Falso	Verdadeira

```
// Exibe a tela de Game Over
if (!game.running) {
    // Instruções...
}
```

37 de 64

▪ Estrutura de controle com múltipla escolha: **switch**

```
switch (variável) {
    case valor_1:
        // Instruções executadas quando a variável possuir o valor 1
        break;
    case valor_2:
        // Instruções executadas quando a variável possuir o valor 2
        break;
    // ...
    default:
        // Instruções executadas quando a variável possuir outro valor
}

// Instruções executadas após o comando switch
```

38 de 64

▪ Estrutura de controle com múltipla escolha: **switch**

```
// Controla o estado do jogador, com base na entrada do usuário
switch (state) {
    case PlayerState::Idle:
        // Instruções executadas quando o jogador estiver parado
        break;
    case PlayerState::Walking:
        // Instruções executadas quando o jogador estiver andando
        break;
    case PlayerState::Jumping:
        // Instruções executadas quando o jogador estiver pulando
        break;
}
```

39 de 64

▪ Laço de repetição com condição inicial: **while**

```
while (condição) {
    // Instruções executadas
    // enquanto a condição
    // for verdadeira
}

// Instruções executadas após o
// término do laço
```

40 de 64

▪ Laço de repetição com condição final: **do-while**

```
do {
    // Instruções executadas
    // até a condição for verdadeira
    // (pelo menos uma vez)
} while (condição);

// Instruções executadas após o
// término do laço
```

41 de 64

▪ Laço de repetição com variável de controle: **for**

```
for (início; fim; incremento) {
    // Instruções executadas
    // enquanto a condição
    // final não for satisfeita
}

// Instruções executadas após o
// término do laço
```

42 de 64

▪ Funções

- As **funções** são blocos de código que realizam tarefas específicas. Elas permitem que o código seja mais modular e reutilizável, facilitando a manutenção e a compreensão do programa.
- As funções em C++ possuem uma definição que inclui o **tipo de retorno**, o **nome da função**, a **lista de parâmetros** e o **corpo da função** que contém o código a ser executado. Se a função não retornar nenhum valor, podemos utilizar a palavra reservada **void**. A lista de parâmetros é opcional e pode conter atributos com valores **default**.
- Trechos repetitivos costumam ser inseridos dentro de uma função, durante a etapa de **refatoração** do código.

43 de 64

▪ Estrutura básica de uma função

```
tipo_de_retorno nome_da_função (parâmetros) {
    // Corpo da função
    // Instruções que devem ser realizadas
    // Valor que deve ser retornado
}
```

44 de 64

▪ Exemplo de uma função

```
#include <iostream>
using namespace std;

// Definição da função
int soma(int a, int b = 0) {
    return a + b;
}

// Função principal
int main() {
    int resultado = soma(5, 3);
    cout << "A soma é: " << resultado << endl;
    return 0;
}
```

45 de 64

Como a função soma() foi definida antes da função principal, não é necessário inserir o protótipo da função.

Chamada da função

Exemplo de uma função

```
#include <iostream>
using namespace std;

// Protótipo da função
int soma(int a, int b = 0);

// Função principal
int main() {
    int resultado = soma(5, 3);
    cout << "A soma é: " << resultado << endl;
    return 0;
}

// Definição da função
int soma(int a, int b = 0) {
    return a + b;
}
```

Como a função soma() foi definida depois da função principal, é necessário inserir o protótipo da função.

Chamada da função

Definição da função

Função com passagem de parâmetro

- Em C++, a passagem de parâmetros para funções pode ser feita de três maneiras principais: por **valor**, por **referência** e por **ponteiro**.
- Passagem por valor**: simples e segura, pois não há risco de alterar o valor original. No entanto, pode ser ineficiente para tipos de dados grandes, pois envolve a cópia dos dados.
- Passagem por referência**: eficiente, pois não há cópia dos dados, mas o valor original pode ser modificado inadvertidamente. Útil quando você deseja que a função modifique o argumento original.
- Passagem por ponteiro**: similar à passagem por referência, mas usada quando o argumento pode ser opcional (permitindo o uso de ponteiros nulos por meio da constante `nullptr`) ou quando a função precisa alterar o endereço do ponteiro.

Função com passagem de parâmetro

- Cada técnica tem a sua indicação de uso e a escolha depende do contexto e das necessidades específicas da função.
- Utilizamos **passagem por valor** quando a função não precisa modificar o argumento original e o tipo de dados é pequeno, como os tipos primitivos.
- Utilizamos **passagem por referência** quando a função precisa modificar o argumento original ou quando o tipo de dados é grande, como classes ou estruturas.
- Utilizamos **passagem por ponteiro** quando a função precisa modificar o argumento original e pode receber um ponteiro nulo, ou quando precisamos manipular diretamente os endereços de memória.

Passagem por valor

- Na passagem por valor, uma cópia do argumento é passada para a função. Modificações feitas ao parâmetro dentro da função não afetam o argumento original.

```
#include <iostream>
using namespace std;

void incrementarPorValor(int num) {
    num++;
    cout << "Dentro da função: " << num << endl;
}

int main() {
    int a = 5;
    incrementarPorValor(a);
    cout << "Fora da função: " << a << endl;
    return 0;
}
```

Dentro da função: 6
Fora da função: 5

Passagem por referência

- Na passagem por referência, uma alias ao argumento original é passado para a função. Modificações feitas ao parâmetro dentro da função afetam o argumento original.

```
#include <iostream>
using namespace std;

void incrementarPorReferencia(int &num) {
    num++;
    cout << "Dentro da função: " << num << endl;
}

int main() {
    int a = 5;
    incrementarPorReferencia(a);
    cout << "Fora da função: " << a << endl;
    return 0;
}
```

Dentro da função: 6
Fora da função: 6

Passagem por ponteiro

- Na passagem por ponteiro, o endereço do argumento é passado para a função. Isso permite que a função modifique diretamente o valor do argumento original.

```
#include <iostream>
using namespace std;

void incrementarPorPonteiro(int *num) {
    (*num)++;
    cout << "Dentro da função: " << *num << endl;
}

int main() {
    int a = 5;
    incrementarPorPonteiro(&a);
    cout << "Fora da função: " << a << endl;
    return 0;
}
```

Dentro da função: 6
Fora da função: 6

Exemplos de funções internas: `cmath`, `cstdlib` e `ctime`

Função	Descrição
<code>pow(x,y)</code>	Retorna o valor da potência da base x , elevado ao expoente y .
<code>sqrt(x)</code>	Retorna o valor da raiz quadrada de x .
<code>ceil(x)</code>	Retorna o valor x , arredondado para cima.
<code>floor(x)</code>	Retorna o valor x , arredondado para baixo.
<code>fmod(x,y)</code>	Retorna o resto da divisão, considerando valores reais.
<code>abs(x)</code>	Retorna o valor absoluto de um número inteiro.
<code>fabs(x)</code>	Retorna o valor absoluto de um número real.
<code>div(x,y)</code>	Retorna o quociente e o resto, resultado da divisão de dois inteiros.
<code>rand()</code>	Retorna um valor pseudoaleatório, entre 0 e 32767.
<code>srand(x)</code>	Retorna um valor pseudoaleatório, considerando o valor da semente x .
<code>exp(x)</code>	Retorna o valor do número de Euler (2.71828), elevado ao expoente x .

Exemplos de funções internas: `cmath`, `cstdlib` e `ctime`

Função	Descrição
<code>log(x)</code>	Retorna o logaritmo natural do valor x .
<code>log10(x)</code>	Retorna o logaritmo na base 10 do valor x .
<code>atof(x)</code>	Retorna o valor do string x , convertido para um float .
<code>atoi(x)</code>	Retorna o valor do string x , convertido para um int .
<code>atol(x)</code>	Retorna o valor do string x , convertido para um long int .
<code>atoll(x)</code>	Retorna o valor do string x , convertido para um long long int .
<code>asctime(x)</code>	Retorna a data e hora, correspondente ao ponteiro x .
<code>ctime(x)</code>	Retorna a data e hora, correspondente ao ponteiro x .
<code>gmtime(x)</code>	Retorna uma estrutura com informações de data e hora, correspondente ao ponteiro x .
<code>mktime(x)</code>	Converte uma estrutura com representada pelo ponteiro x , em uma estrutura adequada para uso com outras funções de tempo.

Sobrecarga de funções

- A **sobrecarga de funções** em C++ é uma característica que permite definir múltiplas funções com o mesmo nome, mas com assinaturas diferentes, dentro do mesmo escopo. A assinatura de uma função inclui o nome da função e o tipo e número de seus parâmetros. Ela permite que diferentes variantes de uma função sejam chamadas dependendo dos argumentos passados na chamada da função.
- O compilador escolhe a versão correta da função a ser chamada com base nos argumentos fornecidos. Isso permite que o mesmo nome de função seja utilizado para operar em diferentes tipos e números de argumentos, facilitando a legibilidade e a manutenção do código.

Sobrecarga de funções

```
#include <iostream>

using namespace std;

// Função sobrecarregada sem parâmetros
void imprime() {
    cout << "Função sem argumentos." << endl;
}

// Função sobrecarregada, com um parâmetro inteiro
void imprime(int valor) {
    cout << "Função com argumento inteiro: " << valor << endl;
}
```

55 de 64



Sobrecarga de funções

```
// Função sobrecarregada, com um parâmetro real
void imprime(float valor) {
    cout << "Função com argumento real: " << valor << endl;
}

// Função sobrecarregada, com dois parâmetros
void imprime(int N1, float N2) {
    cout << "Primeiro valor: " << N1 << endl;
    cout << "Segundo valor: " << N2 << endl;
}
```

56 de 64



Sobrecarga de funções

```
// Função principal
int main() {
    cout << "\n";

    // Utiliza as funções sobrecarregadas
    imprime();
    imprime(10);
    imprime(10.99f);
    imprime(1, 0.99f);

    cout << "\n";

    return 0;
}
```

Modo correto de utilizar um float

Função sem argumentos.
Função com argumento inteiro: 10
Função com argumento real: 10.99
Primeiro valor: 1
Segundo valor: 0.99

57 de 64



Escopo e visibilidade

- Em programação, **escopo** e **visibilidade** são conceitos fundamentais que determinam onde variáveis, funções, classes e outros identificadores podem ser acessados e utilizados no código.
- O **escopo** refere-se à região do código onde um identificador (como uma variável ou função) é válido e pode ser acessado. Em C++, temos os seguintes tipos de escopo: **escopo de bloco**, **escopo de função**, **escopo de arquivo**, **escopo de classe** e **escopo de namespace**.
- A **visibilidade** refere-se à capacidade de acessar um identificador a partir de diferentes partes do código. Ela é controlada por meio de **modificadores de acesso** dentro das classes, **palavras-chave** e **namespaces**.

58 de 64



Tipos de escopo

- Escopo de bloco:** definido por um par de chaves { e }, indica que as variáveis declaradas dentro de um bloco são acessíveis apenas dentro desse bloco.
- Escopo de função:** indica que as variáveis declaradas dentro de uma função são acessíveis apenas dentro dessa função.
- Escopo de arquivo:** indica que as variáveis globais e funções declaradas fora de todas as funções ou classes têm escopo de arquivo, ou seja, acessíveis em qualquer parte do arquivo após sua declaração.
- Escopo de classe:** indica que os membros de uma classe (variáveis e funções) têm escopo dentro da classe, ou seja, podem ser acessados por meio de instâncias da classe.
- Escopo de namespace:** indica que os elementos declarados dentro de um *namespace* têm escopo dentro desse *namespace*.

59 de 64



Tipos de visibilidade

- Modificadores de acesso em classes (*public*, *protected* e *private*):
 - Public:** indica que os membros são acessíveis de qualquer parte do código;
 - Protected:** indica que os membros são acessíveis dentro da própria classe, assim como dentro de classes derivadas e classes amigas;
 - Private:** indica que os membros são acessíveis apenas dentro da própria classe e dentro de classes amigas.
- A palavra-chave **extern** é utilizada para declarar uma variável ou função que foi definida em outro arquivo. Isso permite que diferentes partes do código compartilhem a mesma variável ou função global.
- A palavra-chave **static** permite manter o valor de uma variável, entre as chamadas da função. Em nível de arquivo, ela é visível apenas dentro do arquivo onde foi definida. Dentro de uma classe, membros desse tipo são compartilhados por todas as instâncias da classe.
- A palavra-chave **register** sugere ao compilador que a variável seja armazenada em um registrador da CPU, se possível, para acesso mais rápido. No entanto, o compilador pode ignorar essa sugestão. Seu uso foi descontinuado a partir do C++17.

60 de 64



Namespaces

- Em C++, um espaço de nomes, ou **namespace**, é um mecanismo que permite agrupar identificadores (nomes de variáveis, funções, classes, etc.) sob um nome específico, evitando conflitos de nome e organizando o código de forma mais clara e estruturada. A sua utilização é adequada dentro de grandes projetos, onde múltiplas bibliotecas ou módulos podem definir identificadores que possuem o mesmo nome.
- Para declarar um namespace utilizamos a palavra-chave **namespace**, seguida pelo nome do namespace e um bloco de código delimitado por chaves. Para acessar elementos dentro de um namespace, utilizamos o operador de resolução de escopo, definido por **::**. Para evitar ter que especificar repetidamente o namespace, podemos utilizar a diretiva **using**.

61 de 64



Namespaces

```
// Recursos gráficos
namespace Graphics {
    // Classe Image
    class Image {
        // Instruções
    };

    // Método de desenho
    void draw() {
        // Instruções
    }
}
```

62 de 64



```
// Recursos de áudio
namespace Audio {
    // Classe Sound
    class Sound {
        // Instruções
    };

    // Método de reprodução
    void play() {
        // Instruções
    }
}
```

63 de 64



Namespaces

```
// Função principal
int main() {
    // Instanciamento das classes
    Graphics::Image img;
    Audio::Sound snd;

    // Utilização dos métodos
    Graphics::draw();
    Audio::play();

    // Instruções

    return 0;
}
```

Utilização dos namespaces e do operador de resolução de escopo

63 de 64



- **Na próxima aula veremos...**
 - Revisão de conceitos sobre a Linguagem C++.

